05d — Redis & Memcached

Key Concepts

- **In-memory store** — data kept in RAM, not disk; ~100x faster than disk-based reads
- **Cache hit / miss** — hit = data in Redis, returned instantly; miss = go to DB, store in Redis, return
- **TTL (Time To Live)** — Redis auto-expires keys after set duration
- **Persistence** — Redis can write to disk so data survives restarts
- **Cache warming** — rebuilding cache after restart, either via real traffic or a pre-load script
- **Thundering herd** — many simultaneous cache misses hitting DB at once after cold start

---

Redis vs Memcached

	Redis	**Memcached**
Data types	Strings, Lists, Sets, Hashes, Sorted Sets, Streams	Strings only
Persistence	Yes (RDB + AOF)	No
Pub/Sub	Yes	No
Clustering	Built-in	Client-side
Multithreading	Single-threaded core	Multi-threaded
Industry usage	Dominant standard	Niche / legacy

Rule: Memcached has a slight raw speed edge at extreme scale for pure string caching. Redis wins everywhere else — richer data types, persistence, pub/sub, and more. Redis is the industry standard.

Redis Persistence Modes

Mode	**How**	**Data Loss on Crash**
RDB (snapshot)	Full dataset saved to disk at intervals (e.g., every 5 min)	Writes since last snapshot
AOF (Append Only File)	Every write command logged to disk	Near-zero
No persistence	Pure in-memory — data lost on restart	Everything

For pure caching: no persistence is fine — DB is the source of truth, cache just warms back up.

Redis Use Cases in Production

Use Case	**How Redis handles it**

Use Case	How Redis handles it
Cache	Store DB query results with TTL
Session storage	Store user sessions with auto-expiry
Rate limiting	Increment counter per user per window, block at threshold
Leaderboards	Sorted Sets — score-based ranking, O(log n) insert
Pub/Sub	Real-time notifications, chat
Distributed locks	Prevent race conditions across servers
Job queues	Lists as FIFO queues for background tasks

Cache Warming

- After restart (no persistence), cache is empty — all requests hit DB
- **Organic warming** — real traffic naturally rebuilds cache over time
- **Proactive warming** — startup script pre-loads top N most accessed items
- Prevents **thundering herd** — simultaneous misses overwhelming DB on cold start

---

Industry Examples

- **Twitter** — Redis stores pre-computed home timelines; 800 tweets per user cached
- **GitHub** — Redis for sessions and background job queues (Sidekiq)
- **Uber** — Sorted Sets store driver geolocation scores
- **Stack Overflow** — heavy Redis caching; entire site runs lean because of it
- **Snapchat** — Redis for real-time friend presence and activity feeds

---

Interview Questions

Q: What is Redis and why use it?

> An in-memory data store — data lives in RAM, making reads/writes ~100x faster than disk. Used as a cache to avoid repeated DB queries, and for sessions, pub/sub, leaderboards, rate limiting, and queues.

Q: Redis vs Memcached — which would you choose?

> Redis in almost all cases — it supports richer data types (not just strings), persistence, pub/sub, and clustering. Memcached has a slight speed edge for pure string caching at extreme scale but Redis is the industry standard.

Q: What happens to the Redis cache when the server restarts?

> Depends on persistence config. With RDB, data is restored from the last snapshot (some loss). With AOF, near-zero loss. With no persistence, cache is empty — all requests miss until the cache warms back up.

Q: What are Redis persistence modes?

> RDB = periodic snapshots to disk, fast restore, some data loss. AOF = logs every write, near-zero loss but slower. You can also disable persistence entirely for a pure cache.

Q: What is the thundering herd problem?

> After a cache restart, many requests arrive simultaneously, all miss cache, and all hit the DB at once — causing a DB spike. Prevented by proactive cache warming (pre-loading top data on startup).